

OpenProof project and demo guide

Prepared for the project owner on 27 September 2026. This guide explains the product, gives a practical demo script, and separates observed test results from expected behavior. Use the saved live cases for a reliable presentation, then show a new evidence collection if the network is responsive.

What you built

OpenProof helps someone turn "Did the project do what it said?" into a precise question backed by inspectable evidence. A visitor chooses a contract, chain, historical cutoff, and one observable field. The app collects an evidence preview. A GenLayer Intelligent Contract independently evaluates the fixed question and persists the decision on Bradbury.

There are two modes. **Public promise** requires an accessible source that establishes the exact dated promise. **Contract state** compares a historical value without claiming that anyone promised it. This distinction makes the product useful even when a user only wants to inspect an on-chain fact.

Template	Exact check	Important boundary
Admin and owner control	owner() equals an expected address	Does not cover every role or beneficial control
Treasury restrictions	unlockTime() is at or after a minimum timestamp	Does not prove continuous locking or every withdrawal path
Contract changes	EIP-1967 implementation equals an expected address	Other proxy standards are unsupported

The evidence chains are Ethereum and Base. GenLayer Bradbury, chain 4221, is the decision network. There is no token trading or transfer to the Intelligent Contract; a submitted transaction uses test GEN for network gas.

Where to find it

- Source: <https://github.com/Inclover05/openproof> (private repository).
- Vercel project: <https://vercel.com/inclover05s-projects/openproof>
- Live application: <https://openproof-three.vercel.app>
- Guide download: <https://openproof-three.vercel.app/OpenProof-Project-and-Demo-Guide.pdf>
- Original private site: <https://openproof-evidence.inclover05.chatgpt.site>

The original site remains available with its saved user cases. Only labelled public test cases were seeded into the Vercel database. A copied old Sites case URL does not automatically become a Vercel case URL.

How a case works

- 1. Define the question.** Enter a neutral statement, choose the mode and template, then specify an exact contract address, Ethereum or Base, a past UTC cutoff, and an expected value. Unsupported chains and getters are rejected. Future cutoffs cannot proceed.
- 2. Check the link.** Public promise mode supports direct public X posts, readable Medium articles, GitHub sources, verified explorers, and selected official documentation. "Check link access" tests retrieval before collecting the whole case. The displayed access result is a server observation, not a validator verdict.
- 3. Collect and review.** The server maps the cutoff to a finalized block, verifies that the next block is after the cutoff, and reads the selected field at that block. Each evidence card records its URL, retrieval time, access status, and applicable hash, block, chain, and value. A preview expires after 15 minutes.
- 4. Save.** Saving costs nothing and requires no wallet. Inputs must match the stored evidence snapshot. A repeated save returns the same case. The saved draft has no verdict. "Create revised case" copies the inputs into a new builder while preserving the old record.
- 5. Submit.** Connect a browser wallet to Bradbury. The app requests a live fee estimate, checks the selected account and network again, and asks the wallet to sign the exact payload. The transaction ID is retained locally and on the server so a timeout can be recovered without blindly resubmitting.
- 6. Inspect the decision.** Validators independently retrieve reviewed sources, verify their hashes, check historical contract state, and apply the bounded rubric. The contract uses strict agreement on the required decision fields. The website cannot manufacture the final outcome.

Result	What it means
Supported	The eligible evidence supports this exact comparison
Contradicted	The eligible evidence conflicts with this exact comparison
Insufficient evidence	A required fact could not be established

Lifecycle and execution are separate. An accepted transaction with failed execution is not a successful case. A successful decision can remain in its appeal window before finalization. The UI shows appeal availability when the network returns it; there is no public appeal-signing button in this release.

A five minute demo

Before you start

Open the deployed homepage, the matching-owner live case, the contradictory case, and the unrelated-X case in separate tabs. Keep this guide beside you. If you plan to submit a new case, use a browser wallet funded with test GEN on Bradbury. Never enter a seed phrase or private key into OpenProof. Stop if the wallet presents a security warning.

Your opening words

"OpenProof turns a public promise into a question we can actually check. We choose one contract, one field, and one point in time. You can inspect the evidence before connecting a wallet. GenLayer validators make the decision independently."

Minute one Show the idea

Scroll to "What can you actually prove?" Switch between "The value matches," "The value differs," and "A fact is missing." Explain that this is a labelled interactive illustration. Point out that missing evidence does not mean a promise was broken.

Minute two Show a real result

Open the "Matching owner" live case. Say: "This is a historical contract-state test, not a claim that Circle made a promise." Show the Base contract address, cutoff, block 51,853,326, observed owner address, network transaction, execution result, and current lifecycle. Open the explorer link if useful.

Minute three Change the assertion

Open "Different owner." The same historical observation was compared against the zero address. The outcome is Contradicted. Explain why this finding says nothing about anyone's intent or the safety of a token.

Minute four Show source limitations

Open "Unrelated X source." The post was readable, but it did not establish the submitted contract promise, so GenLayer returned Insufficient evidence. In a fresh builder, paste the Medium link from the source tests and use "Check link access." If blocked, show the corroborating-source field and explain that the original link stays in the record.

Minute five Make a new case

Use "Create revised case" from the matching-owner record. Review the prefilled inputs, collect evidence, and save a new draft. Show that the draft has no verdict. If time permits, review the testnet fee and sign; otherwise finish with the already assessed cases. Do not promise an instant decision or wait silently through a delayed testnet transaction.

Closing words: "The useful output is a transparent record: what was asked, what was accessible, what was observed, and what the network could establish. The limits are part of that record."

Reproducible historical state tests

For all three tests below, choose **Contract state**, **Base**, and the cutoff **2026-09-27 08:00 UTC**. Leave the source blank. Use a neutral statement of at least 25 characters. Collect evidence, inspect the exact value, save, and then submit only if you want a new network judgment. Saving alone must show Draft and no outcome.

Test one Matching owner

- Template: Admin and owner control.
- Contract: 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913
- Field: owner()
- Expected: 0x3abd6f64a422225e61e435bae41db12096106df7
- Statement: Historical comparison test Base USDC owner equals the expected address at the cutoff.
- Expected decision when historical evidence is available: **Supported**.
- Observed: Supported, successful execution, five AGREE votes.
- Saved case: /case/2735f336-d5cf-417c-a267-8f0918455293

Test two Contradictory owner

Repeat test one, changing only the expected address to **0x000000000000000000000000000000000000**. Expected and observed decision: **Contradicted**. The historical owner remains the nonzero address above. This is a comparison test, not an allegation.

Saved case: /case/d7ca8a0d-92cd-4d16-b68d-9ba999205c4d

Test three Proxy implementation

- Template: Contract changes.
- Contract: 0x4200000000000000000000000000000000000000
- Field: EIP-1967.
- Expected: 0xc0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d30010
- Statement: Historical comparison test Base bridge implementation equals the expected address at the cutoff.
- Expected and observed decision: **Supported**, with five AGREE votes.
- Saved case: /case/62321760-00ca-4f50-bf31-eaf3dba2594e

All three observations were retrieved at Base block 51,853,326. Public RPC availability can change. If the required historical value cannot be independently obtained on a later run, a safe insufficient or failure state is appropriate; never substitute the latest value to force the expected result.

Medium and X tests

Public X post

Source: <https://x.com/GenLayer/status/2041643224536592387>

Use "Check link access." Expected: readable text attributed to the X public-post provider, with the original URL, official retrieval endpoint, content hash, and a post-ID timestamp. This exact post was readable in both the server check and the GenLayer probe, which received five AGREE votes.

Now use Public promise mode with the Base USDC address from test one and this statement: "Negative test fixture this unrelated public post does not establish a dated promise about Base USDC ownership." Use the same cutoff and owner field. Expected: **Insufficient evidence**, because a readable post does not establish that specific promise. Observed: SOURCE_NOT_SUFFICIENT, successful execution, four AGREE votes and one TIMEOUT.

Saved case: /case/72f7f19d-e0b1-488f-9ace-a06567b0c37b

Medium article

Source: <https://medium.com/@GenLayer.com/introducing-genlayer-the-intelligence-layer-of-the-internet-03760396c25e>

This real public article was blocked with HTTP 403 in the server experiment. The GenLayer probe also reported blocked, with five AGREE votes and an empty content hash. Expected for the tested access conditions: display the access failure and retain the original URL. Do not invent an excerpt or a verdict about what the article says.

Medium support is conditional: a public article must expose its readable article body. The app does not bypass paywalls or login walls. Page-supplied publication dates are labelled as such. Custom Medium publication domains outside the supported host set are not fetched.

Missing or unsupported post

Try <https://x.com/GenLayer/status/9999999999999999999> and then <https://x.com/GenLayer> . The nonexistent post should produce unavailable or a provider access error, without a hash. The profile URL should be blocked with guidance to use a direct post URL. A valid source URL is not necessarily a valid source for the claim.

Corroboration

Keep a blocked original and add a public official permalink in the separate corroborating-source field. Both references must remain visible. The second source must independently establish the same exact dated promise. A generic explorer page or unrelated GitHub file is not enough. This provenance behavior passed direct fixture tests; a positive live social-promise case was not established during this release.

Functional tests and expected behavior

Test to perform	Expected result
Enter a future cutoff	Validation rejects it before collection
Enter an incomplete contract address	Useful validation error; no submission
Omit the source in Public promise mode	Source required
Omit the source in Contract state mode	Allowed; historical chain evidence still required
Use https://127.0.0.1/admin as a source	Blocked without fetching the private address
Use a redirecting source	Blocked; user must choose the final supported URL
Try an unsupported getter such as <code>admin()</code>	Rejected by the bounded schema
Change inputs after collecting	Collect a fresh snapshot before saving
Save the same snapshot twice	Same case ID; no duplicate record
Reload a saved case	Original inputs and evidence persist
Create a revised case	Prefilled new builder; original record remains intact
Attach an invalid or unrelated transaction	Rejected; exact contract and payload must match
Open without a browser wallet	Read-only features work; signing reports no wallet
Change wallet account or chain before signing	Reconnect and requote before submission
Lose tracking after submission	Resume using the retained transaction ID
View on a 390 pixel phone viewport	Readable controls and cards without horizontal overflow

What was actually checked

Thirty direct contract tests passed across the legacy and new contracts. Eleven JavaScript regression tests passed. Twelve API acceptance checks passed through the running app and Neon, including persistence, snapshot tamper rejection, origin rejection, source access, and live fee quoting. The production build and lint checks passed after the hosting migration fix.

Browser checks covered desktop rendering, the complete collect-review-save journey, mobile live-case display and reload, the interactive outcome explanation, and prefilled revision. No browser errors were reported in those checked journeys. Public production checks and their timestamp are recorded separately in `reports/release.json` and `reports/api-verification.json`.

Fixture tests are not live validator tests. Wallet-extension signing on the final public domain, a public appeal submission, sustained load, and a positive live treasury promise have not been fully exercised. The live transactions here used the funded local testnet signer. Do not describe this release as universally verified or free of all defects.

Technical reference and operating notes

Architecture

The Next.js client provides scope, evidence, review, case pages, and wallet interaction. Server route handlers validate inputs and collect bounded evidence. Neon Postgres stores immutable case inputs, evidence snapshots, and rate-limit counters. GenLayerJS handles fee estimation, submission, and status. The Python Intelligent Contract retrieves reviewed evidence and historical RPC observations independently and persists the outcome.

V2 normalizes X post text and Medium article text before hashing, avoiding irrelevant embed or page-wrapper changes. Raw sources retain their raw-content hash. Changed reviewed content leads to insufficient evidence. URLs use a fixed host policy; redirects are not silently followed. Source bodies are capped at 500 KB and JSON requests at 10 KB.

Contracts and network

- Bradbury chain ID: 4221; wallet chain hex: 0x107d.
- V2 contract: 0xB74B3339695C50C6d16168708e2B22A7D6D72EAB
- Legacy contract: 0x0546Ba4582b7733DB52E3309692BCF7b5B1CAcCe
- SDK: genlayer-js 1.1.8.
- V2 deployment transaction:
0x5ddcafa3a55765f0afe6053e6fae5fbb67294e3db38acac74bc0d62a5374d23b

Legacy records preserve their original contract and byte-for-byte payload format. V2 records explicitly identify protocol version 2. The active address is selected from this version, not from arbitrary user input.

Fees and lifecycle

The isolated v2 deployment balance difference was about 0.001254839 test GEN. The earlier isolated v1 judgment cost about 0.000163473 test GEN. These are observed examples, not fixed prices or a price promise. A fresh quote is obtained before wallet confirmation and expires after two minutes. No real-money funds were used by the test signer.

All four new judgments reached successful finalization during the production status checks. One unrelated-source run had a validator timeout. An earlier v1 negative case is finalized. Refresh the live record to see the current network state; do not infer finality from a displayed outcome alone.

Maintenance and honest limits

Deploying changes

Install dependencies with `npm ci`. Link the existing Vercel project, pull environment variables into ignored `.env.local`, and run `npm run db:migrate` against the intended database. Run `npm test`, `npm run lint`, and `npm run build`. The GitHub main branch is connected to Vercel. Inspect deployment status and verify the production API and browser after any change.

`DATABASE_URL` belongs only in server environment configuration. No private key is needed in Vercel. The local test signer is stored in ignored `.secrets` and must never be uploaded, copied into frontend code, or pasted into the demo guide. Test scripts requiring that signer are separate from the website runtime.

If a demo step fails

- Blocked source: show the access result and add an independently relevant corroborating source, or use Contract state for a different question. Do not silently change the mode of an existing case.
- Historical RPC failure: keep the fixed cutoff and report the missing historical fact. Retry later or use an already assessed case.
- Pending or uncertain transaction: refresh or attach its transaction ID. Check wallet history before any resubmission.
- Storage failure: retain the entered inputs, inspect Vercel environment and Neon availability, and retry collection if the snapshot expired.
- Rate limit: wait five minutes. The app allows ten requests per action per IP window; a shared connection can share that allowance.

Limitations to say out loud

OpenProof is a beta for narrow evidence questions. It is not a universal fact checker, a legal determination, or a security audit. It does not establish identity, motives, beneficial ownership, every admin role, continuous treasury restrictions, or all upgrade mechanisms. Public data providers and testnet services can fail or rate-limit. A current source page is not independent proof that the same text existed before a historical cutoff.

Cases on the public deployment are readable by anyone with the URL; case identifiers are not an authentication system. Do not include private information. Repository source is private by default; give reviewers access or deliberately choose a public release when ready. The old private Sites deployment has not been deleted or exposed by this migration.

Reference links

- GenLayer web access: <https://docs.genlayer.com/developers/intelligent-contracts/features/web-access>
- GenLayer transactions: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions>
- Circle contract addresses: <https://developers.circle.com/stablecoins/usdc-contract-addresses>
- Base contract reference: <https://docs.base.org/specifications/reference/base-contracts>
- GenLayer explorer: <https://explorer-bradbury.genlayer.com>

The repository contains the exact transaction IDs, evidence values, provider observations, and verification scripts. Use those records as the basis of technical claims in a Builder submission. Submitting to the Builder portal is a separate action.